AIORI-3 HACKATHON | CYBERSECURITY

Incident Timeline and Evidence Collection for Small Teams

A Lightweight Digital Forensics and Incident Investigation Tool

TEAM NAME	PROBLEM STATEMENT	TEAM CODE
MATRIX	PS-013	A3-PS013-TC280

Team Members: Ankit Kumar, Janvi Soni

Abstract & Problem Descriptio

Abstract

During a cybersecurity incident, small IT teams often struggle to reconstruct the sequence of events because system and application logs are scattered across multiple sources. Different log formats, time zones and clock differences make manual investigation slow and inconsistent.

We propose a lightweight digital forensics tool that collects host and application logs, preserves original evidence, generates cryptographic hashes, normalizes timestamps to Coordinated Universal Time (UTC), and creates a searchable chronological incident timeline. The system also exports incident summary reports and is evaluated using synthetic incident scenarios.

Problem Description

- Security logsaredistributed across Linux servers, web applications and network devices.
- Different timestamp formats and time zones make event correlation difficult.
- Manual investigation requires considerable time and effort.
- Original evidence must remain unchanged and verifiable during analysis.
- Small IT teams need a consistent way to investigate incidents.

GOAL

Build a tool that helps small teams reconstruct cybersecurity incidents faster while preserving the integrity of collected evidence.

Background & Motivation

Why is incident reconstruction difficult?

Heterogeneous logs Different sources use different formats and field names.	**Timestamp inconsistency** Events may be recorded in different time zones.
Clock skew System clocks may differ from one another or from actual time.	**Evidence integrity** Original records must be preserved during processing.
Investigation complexity Analysts must connect related events across multiple sources.	**Small-team constraints** Limited time and resources make manual analysis harder.

Proposed approach

Combine structuredlog parsing, timestamp normalization, evidence hashing and searchable event visualization in one workflow. The tool organizes events for investigation; it does not automatically prove that every event was caused by an attacker.

Literature Review & Technical Foundation

Digital forensics and incident response

Digitalforensicsemphasizescollectingandpreservingevidence while maintaining integrity. This provides the foundation for the tool's evidence-handling workflow.

Cryptographic hashing

SHA-256producesafixed-lengthdigestofcollecteddata.Comparing a stored hash with a newly calculated hash can help identify whether a file has changed since hashing.

Log normalization and event correlation

Mappingdifferentlogformatsintoasharedstructureandconverting valid timestamps to UTC makes events easier to compare chronologically.

Project motivation

Theprojectaims to providesmall teams with one workflow combining log ingestion, evidence preservation, timestamp normalization, searchable timelines and report export. This is a proposed practical gap, not a claim that existing tools lack these capabilities.

Primary reference

NIST SP 800-86: Guide to Integrating Forensic Techniques into Incident Response

Proposed Solution

Alightweightdigitalforensicsapplicationthattransforms raw log files into a searchable incident timeline while preserving original evidence.

1. Log ingestion & parsing Accept host and application logs; process at least two different formats; map supported records into a common structure.	**2. Evidence preservation & hashing** Preserve original files, calculate SHA-256 hashes and record collection metadata.
3. Timestamp normalization Convert valid timestamps to UTC; identify missing, invalid or ambiguous timestamps; document clock-skew limitations.	**4. Timeline & reporting** Sort events chronologically, search and filter records, and export an incident summary report.

Expected benefit

A consistent investigation workflow with transparent evidence handling and less repetitive manual work. All capabilities will be validated through testing before being claimed as implemented.

Proposed Optimization

Thetoolaimstoreducerepetitivemanualloganalysisbyautomating parsing, timestamp normalization, evidence hashing and event organization.

Metric	Manual baseline	Proposed tool
Log processing	Manual inspection of individual files	Automated parsing of supported formats
Event correlation	Compare timestamps and sources manually	Unified UTC timeline
Evidence integrity	Manual handling and tracking	SHA-256 hash records for verification
Event search	Search individual log files	Searchable centralized timeline
Incident summary	Compile findings manually	Exportable summary report
Performance	Record manual triage time	Measure tool-assisted triage time

Evaluation plan: Use identical synthetic datasets for manual and tool-assisted trials. Record triage time, relevant events identified, parsing failures and hash-verification results. Do not present unmeasured figures as results.

Solution Architecture & Design

Component view of the proposed log investigation tool



Data Flow & Synthetic Test Scenarios

End-to-end workflow

1. INPUT Upload logs in two supported formats.
2. PRESERVE Keep original files unchanged and calculate hashes.
3. PARSE + NORMALIZE Extract fields and convert valid timestamps to UTC.
4. VALIDATE Record invalid timestamps, parsing failures and clock-skew limitations.
5. ANALYZE Create a chronological, searchable event timeline.
6. OUTPUT Export report with relevant events and evidence metadata.

Three synthetic incident scenarios

- **Scenario 1 — Suspiciousauthentication:** multiple failed logins followed by a successful login.
- **Scenario 2 — Web application incident:** unusual requests followed by application errors.
- **Scenario 3 — Cross-source correlation:** related events in two formats with different time zones and a simulated clock offset.

Each scenario will have a known expected sequence for objective evaluation. Use synthetic logs only.

Implementation Timeline & Evaluation

Five-week development plan

Week 1	Project setup	Define log formats, synthetic datasets and original-evidence workflow.
Week 2	Parser and hashing	Implement two parsers, SHA-256 calculation and UTC normalization.
Week 3	Timeline dashboard	Add chronological sorting, event search, filters and structured storage.
Week 4	Integration and reports	Integrate modules, export reports and document parsing gaps.
Week 5	Testing and demo	Run three scenarios, compare triage time and prepare the demonstration.

Evaluation metrics

- Triage time:time requiredto reconstruct each incident.
- Parsing coverage: proportion of records parsed successfully.
- Event ordering: comparison with the known expected sequence.
- Evidence integrity: hash verification after collection.
- Parsing gaps: count and description of unsupported or invalid records.

Prototype Pseudocode & References

Illustrative pseudocode

```
FOR each uploaded log file:
calculate SHA-256 hash
preserve original file
parse records using the matching parser
FOR each record:
normalize valid timestamp to UTC
if parsing or timestamp fails:
record the parsing gap
else:
add structured event to timeline
sort events by normalized timestamp
export report with events, evidence hashes and parsing gaps
```

References

1. National Institute of Standards and Technology (NIST), SP 800-86, Guide to Integrating Forensic Techniques into Incident Response. https://csrc.nist.gov/pubs/sp/800/86/final

2. Python Software Foundation, Python Documentation. https://docs.python.org/3/

3. Python Software Foundation, hashlib — Secure hashes and message digests. https://docs.python.org/3/library/hashlib.html

4. SQLite Documentation. https://www.sqlite.org/docs.html

THANK YOU

Making Cybersecurity Incident Investigation
Faster, Clearer and More Reliable

TEAM MATRIX
PS-013 • A3-PS013-TC280 • AIORI-3 Hackathon
Questions & Discussion